

04_metrics-py

#python #api

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
├── backend/  
│   ├── app/  
│   │   ├── api/  
│   │   │   ├── routes/  
│   │   │   └── metrics.py
```

El archivo `metrics.py` se encuentra dentro de la carpeta de rutas del backend:

```
backend/app/api/routes/
```

Este archivo define el endpoint encargado de devolver métricas agregadas del laboratorio SIEM MVP.

A diferencia de `health.py`, que solo comprueba el estado de la API y la base de datos, o `info.py`, que devuelve información del backend, `metrics.py` sí consulta datos reales almacenados en PostgreSQL.

Este router se importa y registra en `backend/app/main.py` mediante estas líneas:

```
from app.api.routes.metrics import router as metrics_router
```

```
app.include_router(metrics_router)
```

Gracias a esto, el endpoint definido en `metrics.py` queda incorporado a la aplicación FastAPI principal.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,260p' backend/app/api/routes/metrics.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,260p'
```

Indica que se impriman las líneas de la 1 a la 260.

```
backend/app/api/routes/metrics.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```

from __future__ import annotations

from fastapi import APIRouter, Depends, Query
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule

router = APIRouter(prefix="/metrics", tags=["metrics"])

@router.get("")
def get_metrics(
    top_groups: int = Query(10, ge=1, le=100),
    db: Session = Depends(get_db),
):
    events_total = db.execute(select(func.count()).select_from(Event)).scalar_one()
    rules_total = db.execute(select(func.count()).select_from(Rule)).scalar_one()
    rules_enabled = db.execute(
        select(func.count()).select_from(Rule).where(Rule.enabled.is_(True))
    ).scalar_one()
    alerts_total = db.execute(select(func.count()).select_from(Alert)).scalar_one()

    rows_status = db.execute(
        select(Alert.status, func.count())
        .group_by(Alert.status)
        .order_by(func.count().desc())
    ).all()
    alerts_by_status = {status: count for status, count in rows_status}

    rows_group = db.execute(
        select(Alert.group_key, func.count())
        .where(Alert.group_key.is_not(None))
        .group_by(Alert.group_key)
        .order_by(func.count().desc())
        .limit(top_groups)
    ).all()
    alerts_by_group_key = {group_key: count for group_key, count in rows_group}

    return {
        "events_total": events_total,
        "rules_total": rules_total,
        "rules_enabled": rules_enabled,
        "alerts_total": alerts_total,
        "alerts_by_status": alerts_by_status,
        "alerts_by_group_key_top": alerts_by_group_key,
    }

```

4 Función general del archivo

El archivo `metrics.py` define un endpoint de métricas del laboratorio.

La ruta expuesta es:

```
GET /metrics
```

Este endpoint consulta la base de datos y devuelve un resumen del estado del sistema.

Las métricas que devuelve son:

events_total	→ número total de eventos almacenados
rules_total	→ número total de reglas creadas
rules_enabled	→ número de reglas activas
alerts_total	→ número total de alertas generadas
alerts_by_status	→ alertas agrupadas por estado
alerts_by_group_key_top	→ alertas agrupadas por group_key, limitadas por top_groups

Respuesta esperada aproximada:

```
{
  "events_total": 25,
  "rules_total": 4,
  "rules_enabled": 3,
  "alerts_total": 7,
  "alerts_by_status": {
    "open": 5,
    "closed": 2
  },
  "alerts_by_group_key_top": {
    "192.168.1.10": 3,
    "admin": 2
  }
}
```

Este endpoint es útil porque ofrece una visión agregada del laboratorio sin tener que consultar manualmente eventos, reglas y alertas por separado.

5 Estructura general del archivo

El archivo puede dividirse en siete bloques:

```
from __future__ import annotations
```

Importación futura para anotaciones modernas.

```
from fastapi import APIRouter, Depends, Query
from sqlalchemy import func, select
from sqlalchemy.orm import Session
```

Importaciones externas de FastAPI y SQLAlchemy.

```
from app.db.session import get_db
from app.models.alert import Alert
from app.models.event import Event
from app.models.rule import Rule
```

Importaciones internas del proyecto: sesión de base de datos y modelos.

```
router = APIRouter(prefix="/metrics", tags=["metrics"])
```

Creación del router para /metrics.

```
@router.get("")
def get_metrics(...):
```

Definición del endpoint GET /metrics.

```
events_total = ...
rules_total = ...
```

```
rules_enabled = ...
alerts_total = ...
```

Consultas de conteo global.

```
rows_status = ...
rows_group = ...
return {...}
```

Consultas agrupadas y respuesta final.

Visualmente:

```
metrics.py
├─ Importación futura
├─ Importaciones FastAPI
├─ Importaciones SQLAlchemy
├─ Importaciones internas del proyecto
├─ Creación del router /metrics
├─ Endpoint GET /metrics
├─ Consultas de conteo
├─ Consultas agrupadas
└─ Respuesta JSON
```

6 Análisis línea por línea

Importación futura de anotaciones

```
from __future__ import annotations
```

Esta línea activa un comportamiento moderno de Python relacionado con las anotaciones de tipos.

Desglose:

```
from __future__
```

`__future__` es un módulo especial que permite activar características nuevas o futuras del lenguaje.

```
import annotations
```

Activa el tratamiento diferido de anotaciones.

En este archivo se usan anotaciones como:

```
top_groups: int
db: Session
```

Esta importación ayuda a que las anotaciones se manejen de forma más flexible.

Importación de FastAPI

```
from fastapi import APIRouter, Depends, Query
```

Esta línea importa tres elementos desde FastAPI:

```
APIRouter
Depends
Query
```

APIRouter

`APIRouter` permite crear un grupo de rutas separado de la aplicación principal.

En este archivo se usa aquí:

```
router = APIRouter(prefix="/metrics", tags=["metrics"])
```

Después, `main.py` incorpora este router a la aplicación principal:

```
app.include_router(metrics_router)
```

Depends

`Depends` permite usar el sistema de inyección de dependencias de FastAPI.

En este archivo se usa para obtener una sesión de base de datos:

```
db: Session = Depends(get_db)
```

Esto significa que FastAPI ejecuta `get_db` y proporciona el resultado al parámetro `db`.

Query

`Query` permite definir y validar parámetros recibidos por la URL.

En este archivo se usa aquí:

```
top_groups: int = Query(10, ge=1, le=100)
```

Esto define un parámetro de consulta llamado `top_groups`.

Ejemplo de uso:

```
GET /metrics?top_groups=5
```

Importación de funciones SQLAlchemy

```
from sqlalchemy import func, select
```

Esta línea importa dos elementos importantes de SQLAlchemy:

```
func
select
```

func

`func` permite usar funciones SQL desde SQLAlchemy.

En este archivo se usa principalmente para contar registros:

```
func.count()
```

Esto se traduce conceptualmente a SQL como:

```
COUNT(*)
```

select

`select` permite construir consultas SQL de tipo `SELECT` usando la sintaxis moderna de SQLAlchemy.

Ejemplo del archivo:

```
select(func.count()).select_from(Event)
```

Conceptualmente equivale a:

```
SELECT COUNT(*) FROM events;
```

El nombre real de la tabla dependerá de cómo esté definido el modelo `Event`.

Importación de `Session`

```
from sqlalchemy.orm import Session
```

Esta línea importa la clase `Session` del ORM de SQLAlchemy.

Una sesión representa una conexión lógica de trabajo con la base de datos.

A través de una sesión se pueden ejecutar consultas, insertar datos, actualizar registros o eliminar información.

En este archivo se usa como anotación de tipo:

```
db: Session = Depends(get_db)
```

Esto indica que `db` será una sesión SQLAlchemy.

Importación de `get_db`

```
from app.db.session import get_db
```

Esta línea importa la función `get_db` desde:

```
backend/app/db/session.py
```

`get_db` proporciona una sesión de base de datos al endpoint.

En FastAPI, esta función se utiliza como dependencia:

```
db: Session = Depends(get_db)
```

Su función habitual es:

1. Crear o abrir una sesión de base de datos.
2. Entregarla al endpoint.
3. Cerrarla al terminar la petición.

De esta forma, cada endpoint no tiene que gestionar manualmente la conexión.

Importación del modelo `Alert`

```
from app.models.alert import Alert
```

Esta línea importa el modelo SQLAlchemy `Alert` desde:

```
backend/app/models/alert.py
```

El modelo `Alert` representa la tabla de alertas en la base de datos.

En este archivo se utiliza para contar alertas y agruparlas por estado o por `group_key` .

Ejemplos:

```
select(func.count()).select_from(Alert)
```

```
select(Alert.status, func.count())
```

```
select(Alert.group_key, func.count())
```

Importación del modelo `Event`

```
from app.models.event import Event
```

Esta línea importa el modelo SQLAlchemy `Event` desde:

```
backend/app/models/event.py
```

El modelo `Event` representa la tabla de eventos.

En este archivo se utiliza para contar el número total de eventos almacenados:

```
events_total = db.execute(select(func.count()).select_from(Event)).scalar_one()
```

Importación del modelo `Rule`

```
from app.models.rule import Rule
```

Esta línea importa el modelo SQLAlchemy `Rule` desde:

```
backend/app/models/rule.py
```

El modelo `Rule` representa la tabla de reglas.

En este archivo se utiliza para contar:

- Total de reglas.
- Total de reglas activas.

Ejemplos:

```
select(func.count()).select_from(Rule)
```

```
select(func.count()).select_from(Rule).where(Rule.enabled.is_(True))
```

Creación del router

```
router = APIRouter(prefix="/metrics", tags=["metrics"])
```

Esta línea crea un router de FastAPI para las métricas.

Desglose:

```
router
```

Variable donde se guarda el router.

Debe llamarse `router` porque en `main.py` se importa así:

```
from app.api.routes.metrics import router as metrics_router
```

```
APIRouter(...)
```

Crea una instancia de router.

Parámetro `prefix`

```
prefix="/metrics"
```

Define el prefijo común para todas las rutas de este archivo.

Como más abajo se usa:

```
@router.get("")
```

la ruta final será:

```
GET /metrics
```

Parámetro `tags`

```
tags=["metrics"]
```

Agrupar este endpoint bajo la etiqueta `metrics` en la documentación Swagger.

La documentación automática está disponible normalmente en:

```
http://localhost:8000/docs
```

Decorador del endpoint

```
@router.get("")
```

Esta línea registra una ruta HTTP de tipo `GET`.

Desglose:

```
@
```

Indica que se está usando un decorador.

```
router
```

Es el router creado anteriormente.

```
.get
```

Indica que el endpoint responderá al método HTTP `GET`.

```
("")
```

Define la ruta relativa dentro del router.

Como el router ya tiene `prefix="/metrics"`, la ruta final será:


```
GET /metrics
```

Definición de la función `get_metrics`

```
def get_metrics(
```

Esta línea inicia la definición de la función que se ejecutará cuando se llame al endpoint `/metrics`.

Desglose:

```
def
```

Palabra clave de Python para definir una función.

```
get_metrics
```

Nombre de la función.

```
(
```

Abre la lista de parámetros.

La función está escrita en varias líneas porque recibe más de un parámetro y se busca legibilidad.

Parámetro `top_groups`

```
top_groups: int = Query(10, ge=1, le=100),
```

Este parámetro permite controlar cuántos grupos de alertas por `group_key` se devuelven.

Ejemplo:

```
GET /metrics?top_groups=5
```

Desglose:

```
top_groups
```

Nombre del parámetro.

FastAPI lo interpreta como un parámetro de query porque no forma parte de la ruta y está definido con `Query`.

```
: int
```

Anotación de tipo. Indica que debe ser un número entero.

```
= Query(...)
```

Define validación y valor por defecto mediante FastAPI.

Valor por defecto `10`

```
Query(10, ge=1, le=100)
```

El primer argumento, `10`, indica el valor por defecto.

Si el usuario llama a:

```
GET /metrics
```

sin indicar `top_groups` , se usará:

```
top_groups = 10
```

Validación `ge=1`

```
ge=1
```

`ge` significa:

```
greater or equal
```

Es decir:

```
mayor o igual que 1
```

Esto impide que el usuario pida cero o un número negativo de grupos.

Validación `le=100`

```
le=100
```

`le` significa:

```
less or equal
```

Es decir:

```
menor o igual que 100
```

Esto impide pedir un número excesivamente alto de grupos.

Esta validación protege el endpoint frente a consultas demasiado grandes.

Parámetro `db`

```
db: Session = Depends(get_db),
```

Este parámetro proporciona la sesión de base de datos.

Desglose:

```
db
```

Nombre del parámetro.

```
: Session
```

Anotación de tipo. Indica que `db` será una sesión SQLAlchemy.

```
= Depends(get_db)
```

Indica que FastAPI debe ejecutar `get_db` para obtener el valor de `db` .

Esto conecta el endpoint con la base de datos PostgreSQL.

Cierre de la firma de la función

```
) :
```

Esta línea cierra los parámetros de la función.

El carácter `:` indica que empieza el bloque de código de la función.

Todo lo que esté indentado debajo pertenece a `get_metrics`.

Conteo total de eventos

```
events_total = db.execute(select(func.count()).select_from(Event)).scalar_one()
```

Esta línea calcula el número total de eventos almacenados.

Desglose:

```
events_total
```

Variable donde se guarda el resultado.

```
db.execute(...)
```

Ejecuta una consulta SQLAlchemy usando la sesión de base de datos.

```
select(func.count())
```

Construye una consulta `SELECT COUNT(*)`.

```
.select_from(Event)
```

Indica que el conteo se realiza sobre la tabla asociada al modelo `Event`.

Conceptualmente equivale a:

```
SELECT COUNT(*) FROM events;
```

```
.scalar_one()
```

Extrae un único valor escalar del resultado.

Como `COUNT(*)` devuelve una sola fila y una sola columna, `scalar_one()` obtiene directamente ese número.

Resultado esperado:

```
events_total = 25
```

Conteo total de reglas

```
rules_total = db.execute(select(func.count()).select_from(Rule)).scalar_one()
```

Esta línea calcula el número total de reglas almacenadas.

Es equivalente a la anterior, pero usando el modelo `Rule`.

Conceptualmente equivale a:

```
SELECT COUNT(*) FROM rules;
```

Desglose:

```
rules_total
```

Variable donde se guarda el número de reglas.

```
select(func.count()).select_from(Rule)
```

Cuenta todos los registros de la tabla asociada al modelo `Rule`.

```
.scalar_one()
```

Devuelve el número como valor simple.

Conteo de reglas activas

```
rules_enabled = db.execute(
    select(func.count()).select_from(Rule).where(Rule.enabled.is_(True))
).scalar_one()
```

Este bloque calcula cuántas reglas están activas.

La consulta está dividida en varias líneas para mejorar la legibilidad.

Desglose:

```
rules_enabled
```

Variable donde se guarda el número de reglas activas.

```
db.execute(...)
```

Ejecuta la consulta.

```
select(func.count())
```

Cuenta registros.

```
.select_from(Rule)
```

Indica que el conteo se hace sobre el modelo `Rule`.

```
.where(Rule.enabled.is_(True))
```

Añade una condición.

Solo cuenta las reglas cuyo campo `enabled` sea verdadero.

Conceptualmente equivale a:

```
SELECT COUNT(*) FROM rules WHERE enabled IS TRUE;
```

Uso de `.is_(True)`

```
Rule.enabled.is_(True)
```

En SQLAlchemy, para comparar columnas booleanas con `True`, es habitual usar `.is_(True)`.

Esto genera una comparación SQL adecuada:

```
enabled IS TRUE
```

Es más explícito que usar:

```
Rule.enabled == True
```

y evita algunas advertencias o malas prácticas de estilo.

Conteo total de alertas

```
alerts_total = db.execute(select(func.count()).select_from(Alert)).scalar_one()
```

Esta línea calcula el número total de alertas almacenadas.

Conceptualmente equivale a:

```
SELECT COUNT(*) FROM alerts;
```

Desglose:

```
alerts_total
```

Variable donde se guarda el resultado.

```
select(func.count()).select_from(Alert)
```

Cuenta todos los registros de la tabla asociada al modelo `Alert`.

```
.scalar_one()
```

Extrae el número como valor simple.

Consulta de alertas agrupadas por estado

```
rows_status = db.execute(
    select(Alert.status, func.count())
    .group_by(Alert.status)
    .order_by(func.count().desc())
).all()
```

Este bloque calcula cuántas alertas hay por cada estado.

Por ejemplo:

```
open    → 5
closed  → 2
```

Desglose:

```
rows_status
```

Variable donde se guarda el resultado de la consulta.

```
db.execute(...)
```

Ejecuta la consulta con SQLAlchemy.

```
select(Alert.status, func.count())
```

Selecciona dos valores:

1. El estado de la alerta.
2. El número de alertas con ese estado.

Conceptualmente:

```
SELECT status, COUNT(*)  
FROM alerts  
GROUP BY status  
ORDER BY COUNT(*) DESC;
```

Agrupación por estado

```
.group_by(Alert.status)
```

Agrupar los registros por el campo `status`.

Esto permite contar cuántas alertas hay en cada estado.

Sin `GROUP BY`, no se podría obtener un conteo separado para cada estado.

Orden descendente

```
.order_by(func.count().desc())
```

Ordena los resultados por el número de alertas, de mayor a menor.

Desglose:

```
func.count()
```

Representa el conteo.

```
.desc()
```

Aplica orden descendente.

Así, el estado con más alertas aparece primero.

Obtención de todos los resultados

```
).all()
```

`.all()` obtiene todas las filas devueltas por la consulta.

El resultado tendrá una forma parecida a:

```
[  
    ("open", 5),  
    ("closed", 2)  
]
```

Construcción del diccionario `alerts_by_status`

```
alerts_by_status = {status: count for status, count in rows_status}
```

Esta línea transforma la lista de filas en un diccionario.

Desglose:

```
alerts_by_status
```

Variable donde se guarda el diccionario final.

```
{status: count for status, count in rows_status}
```

Esto es una comprensión de diccionario.

Recorre cada par:

```
status, count
```

dentro de:

```
rows_status
```

y construye un diccionario con esta forma:

```
{
    "open": 5,
    "closed": 2
}
```

Es decir, convierte:

```
[("open", 5), ("closed", 2)]
```

en:

```
{"open": 5, "closed": 2}
```

Este formato es más cómodo para devolverlo como JSON.

Consulta de alertas agrupadas por `group_key`

```
rows_group = db.execute(
    select(Alert.group_key, func.count())
    .where(Alert.group_key.is_not(None))
    .group_by(Alert.group_key)
    .order_by(func.count().desc())
    .limit(top_groups)
).all()
```

Este bloque calcula cuántas alertas hay por cada `group_key`.

El campo `group_key` sirve para agrupar alertas según algún criterio definido por el sistema.

Por ejemplo, puede representar:

- Una IP de origen.
- Un usuario.
- Un tipo de evento.
- Una clave de agrupación generada por una regla.

Depende de cómo se haya diseñado el motor de reglas y alertas.

Selección de campos

```
select(Alert.group_key, func.count())
```

Selecciona:

1. El valor de `group_key`.
2. El número de alertas asociadas a ese `group_key`.

Conceptualmente:

```
SELECT group_key, COUNT(*)  
FROM alerts  
...
```

Filtrado de valores nulos

```
.where(Alert.group_key.is_not(None))
```

Añade una condición para excluir alertas cuyo `group_key` sea nulo.

Desglose:

```
Alert.group_key
```

Columna del modelo `Alert`.

```
.is_not(None)
```

Genera una condición SQL similar a:

```
group_key IS NOT NULL
```

Esto evita que aparezca una entrada con clave nula en el resultado.

Agrupación por `group_key`

```
.group_by(Alert.group_key)
```

Agrupar las alertas por el campo `group_key`.

Esto permite saber qué grupos concentran más alertas.

Orden por número de alertas

```
.order_by(func.count().desc())
```

Ordena los grupos por número de alertas, de mayor a menor.

Así, los grupos más relevantes aparecen primero.

Limitación de resultados

```
.limit(top_groups)
```

Limita el número de grupos devueltos.

El valor de `top_groups` viene del parámetro de query definido en la función:


```
top_groups: int = Query(10, ge=1, le=100)
```

Por defecto, devuelve los 10 grupos principales.

Ejemplo:

```
GET /metrics?top_groups=5
```

devolvería solo los 5 grupos con más alertas.

Obtención de resultados

```
).all()
```

Obtiene todas las filas resultantes después de aplicar agrupación, orden y límite.

El resultado podría tener una forma similar a:

```
[
    ("192.168.1.10", 3),
    ("admin", 2)
]
```

Construcción del diccionario `alerts_by_group_key`

```
alerts_by_group_key = {group_key: count for group_key, count in rows_group}
```

Esta línea transforma los resultados agrupados por `group_key` en un diccionario.

Desglose:

```
alerts_by_group_key
```

Variable donde se guarda el diccionario.

```
{group_key: count for group_key, count in rows_group}
```

Comprensión de diccionario.

Convierte una lista como:

```
[
    ("192.168.1.10", 3),
    ("admin", 2)
]
```

en:

```
{
    "192.168.1.10": 3,
    "admin": 2
}
```

Esto permite devolver el resultado en formato JSON de forma clara.

Inicio de la respuesta final

```
return {
```

Esta línea inicia el diccionario que se devolverá como respuesta del endpoint.

FastAPI convertirá automáticamente este diccionario en JSON.

Campo `events_total`

```
"events_total": events_total,
```

Añade a la respuesta el número total de eventos almacenados.

El valor procede de esta consulta previa:

```
events_total = db.execute(select(func.count()).select_from(Event)).scalar_one()
```

Campo `rules_total`

```
"rules_total": rules_total,
```

Añade a la respuesta el número total de reglas existentes.

El valor procede de:

```
rules_total = db.execute(select(func.count()).select_from(Rule)).scalar_one()
```

Campo `rules_enabled`

```
"rules_enabled": rules_enabled,
```

Añade a la respuesta el número de reglas activas.

El valor procede de:

```
rules_enabled = db.execute(
    select(func.count()).select_from(Rule).where(Rule.enabled.is_(True))
).scalar_one()
```

Este campo permite comparar cuántas reglas existen frente a cuántas están realmente habilitadas.

Campo `alerts_total`

```
"alerts_total": alerts_total,
```

Añade a la respuesta el número total de alertas generadas.

El valor procede de:

```
alerts_total = db.execute(select(func.count()).select_from(Alert)).scalar_one()
```

Campo `alerts_by_status`

```
"alerts_by_status": alerts_by_status,
```

Añade a la respuesta un diccionario con alertas agrupadas por estado.

Ejemplo:

```
"alerts_by_status": {  
  "open": 5,  
  "closed": 2  
}
```

Este campo permite ver la distribución de alertas según su estado operativo.

Campo `alerts_by_group_key_top`

```
"alerts_by_group_key_top": alerts_by_group_key,
```

Añade a la respuesta el ranking de `group_key` con más alertas.

Ejemplo:

```
"alerts_by_group_key_top": {  
  "192.168.1.10": 3,  
  "admin": 2  
}
```

El sufijo `top` indica que no necesariamente devuelve todos los grupos, sino los principales según el límite `top_groups`.

Cierre de la respuesta

```
}
```

Cierra el diccionario devuelto por la función.

Resultado final del archivo

Este archivo expone el endpoint:

```
GET /metrics
```

Su comportamiento es:

1. FastAPI recibe una petición `GET /metrics`.
2. Valida el parámetro opcional `top_groups`.
3. Obtiene una sesión de base de datos mediante `get_db`.
4. Cuenta eventos totales.
5. Cuenta reglas totales.
6. Cuenta reglas activas.
7. Cuenta alertas totales.
8. Agrupa alertas por estado.
9. Agrupa alertas por `group_key`.
10. Devuelve un JSON con las métricas agregadas.

Respuesta esperada aproximada:

```
{  
  "events_total": 25,  
  "rules_total": 4,  
  "rules_enabled": 3,  
  "alerts_total": 7,  
  "alerts_by_status": {  
    "open": 5,  
    "closed": 2  
  },  
  "alerts_by_group_key_top": {
```

```
"192.168.1.10": 3,  
"admin": 2  
}  
}
```

7 Relación con el flujo técnico del laboratorio

`metrics.py` actúa como endpoint de resumen del estado interno del SIEM.

No introduce nuevos eventos ni genera alertas, pero consulta los datos ya generados por otros módulos.

La relación técnica sería:

```
Eventos  
  ↓  
se almacenan en PostgreSQL  
  ↓  
se consultan desde metrics.py  
  ↓  
events_total  
  
Reglas  
  ↓  
se almacenan en PostgreSQL  
  ↓  
se consultan desde metrics.py  
  ↓  
rules_total / rules_enabled  
  
Alertas  
  ↓  
se generan desde el motor de reglas  
  ↓  
se almacenan en PostgreSQL  
  ↓  
se consultan desde metrics.py  
  ↓  
alerts_total / alerts_by_status / alerts_by_group_key_top
```

Dentro del flujo general del laboratorio:

```
Ingesta de eventos  
  ↓  
Persistencia en base de datos  
  ↓  
Evaluación de reglas  
  ↓  
Generación de alertas  
  ↓  
Consulta de métricas
```

Este endpoint permite obtener una visión global del sistema sin revisar cada tabla por separado.

8 Errores típicos o puntos importantes

Error de conexión con la base de datos

Este archivo depende directamente de la sesión de base de datos:

```
db: Session = Depends(get_db)
```

Si PostgreSQL no está disponible, las consultas fallarán.

Causas posibles:

- Contenedor siem-db apagado.
- DATABASE_URL incorrecta.
- Migraciones no aplicadas.
- Tablas no creadas.
- Problemas de red entre api y db.

Error si no existen las tablas

Este endpoint consulta los modelos:

```
Event
Rule
Alert
```

Si las tablas correspondientes no existen en PostgreSQL, las consultas fallarán.

En ese caso habría que revisar las migraciones de Alembic:

```
docker exec -it siem-api alembic current
docker exec -it siem-api alembic upgrade head
```

Validación de `top_groups`

El parámetro:

```
top_groups: int = Query(10, ge=1, le=100)
```

impide valores fuera del rango permitido.

Ejemplos:

```
GET /metrics?top_groups=0    → error de validación
GET /metrics?top_groups=101 → error de validación
GET /metrics?top_groups=5    → correcto
```

FastAPI devolverá un error `422 Unprocessable Entity` si el valor no cumple las restricciones.

Diferencia entre `.scalar_one()` y `.all()`

En el archivo se usan dos formas de obtener resultados:

```
.scalar_one()
```

Se usa cuando se espera un único valor, como un `COUNT(*)`.

```
.all()
```

Se usa cuando se esperan varias filas, como al agrupar por estado o por `group_key`.

Uso de `group_key`

El campo `group_key` permite agrupar alertas por una clave común.

Esto es útil para detectar concentración de alertas sobre un mismo elemento, por ejemplo una IP, usuario o patrón de comportamiento.

El endpoint excluye valores nulos con:

```
.where(Alert.group_key.is_not(None))
```

Así evita devolver grupos sin clave definida.

Métricas como endpoint de lectura

Este endpoint solo consulta datos.

No modifica eventos, reglas ni alertas.

Es decir, su función es de lectura y resumen.

9 Comandos útiles relacionados

Comprobar el endpoint desde el host:

```
curl http://localhost:8000/metrics
```

Comprobar usando un límite distinto de grupos:

```
curl "http://localhost:8000/metrics?top_groups=5"
```

Comprobar desde navegador:

```
http://localhost:8000/metrics
```

Comprobar Swagger:

```
http://localhost:8000/docs
```

Ver logs de la API:

```
docker logs siem-api
```

Ver logs en tiempo real:

```
docker logs -f siem-api
```

Comprobar que la API puede importar el router:

```
docker exec -it siem-api python -c "from app.api.routes.metrics import router; print(router)"
```

Comprobar que la app completa carga correctamente:

```
docker exec -it siem-api python -c "from app.main import app; print(app.title)"
```

Comprobar tablas en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "\dt"
```

Consultar número de eventos directamente en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT COUNT(*) FROM events;"
```

Consultar número de reglas directamente en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT COUNT(*) FROM rules;"
```

Consultar número de alertas directamente en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT COUNT(*) FROM alerts;"
```

Consultar alertas por estado directamente en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT status, COUNT(*) FROM alerts GROUP BY status ORDER BY COUNT(*) DESC;"
```

Consultar alertas por `group_key` directamente en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT group_key, COUNT(*) FROM alerts WHERE group_key IS NOT NULL GROUP BY group_key ORDER BY COUNT(*) DESC LIMIT 10;"
```

Dataview (inline field 'Query(...)'): Error:

-- PARSING FAILED -----

```
> 1 | Query(...)
    |      ^
```

Expected one of the following:

'(', ')', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable

Dataview (for inline query '= Depends(get_db)'): Unrecognized function name 'Depends'